

Pensamiento Computacional aplicado al Proyecto Final

1. Abstracción

- ¿Qué se solicita finalmente? (Problema): Desarrollar una plataforma integral de software para la gestión de eventos y la venta de entradas, que soporte dos perfiles de usuario (Usuario final y Administrador de operaciones).
- **¿Qué información es relevante?: * Entidades principales:** Usuario, Evento, Recinto, Zona, Asiento, Compra, Entrada e Incidencia.
 - **Estados transaccionales:** Los eventos tienen estados (Borrador, Publicado, Pausado, Cancelado, Finalizado) ; los asientos cambian de disponibilidad (Disponible, Reservado, Vendido, Bloqueado) ; y las compras fluyen por diferentes fases (Creada, Pagada, Confirmada, Cancelada, Reembolsada, Incidencia).
 - **Servicios adicionales:** Elementos que modifican el valor de la compra como acceso VIP, seguro, merchandising y parqueadero.
- **¿Cómo se agrupa la información relevante?:**
 - *Jerarquía de Infraestructura:* Un Recinto contiene Zonas , y una Zona contiene Asientos numerados (si aplica).
 - *Agrupación Comercial:* Un Evento ocurre en un Recinto. Una Compra agrupa Entradas (ítems) y servicios adicionales, y se asocia a un Usuario y a un Evento.
- **¿Qué funcionalidades se solicitan finalmente?:**
 - Flujo de cliente: Explorar eventos con filtros, seleccionar asientos, agregar servicios, pagar, gestionar cancelaciones y ver historial/comprobantes.
 - Flujo de administrador: CRUD de entidades (usuarios, eventos, recintos, zonas, asientos), gestión de incidencias y visualización de métricas/reportes.

2. Descomposición

- **¿Cómo se distribuyen las funcionalidades?:** El sistema monolítico se puede dividir en módulos lógicos para facilitar su desarrollo:
 - **Módulo de Inventario y Catálogo:** Gestiona la creación de Recintos, Zonas, Asientos y Eventos.
 - **Módulo de Usuarios y Autenticación:** Maneja el registro, inicio de sesión y perfiles (tanto de clientes como del personal de operaciones).
 - **Módulo de Transacciones (Carrito/Compra):** Controla el ciclo de vida de la compra, la selección de asientos, la adición de servicios extra y la generación de

entradas.

- **Módulo de Operaciones y Reportes:** Encargado del registro de incidencias, exportación de datos (CSV/PDF) y generación de gráficos estadísticos (JavaFX Charts).

3. Reconocimiento de Patrones

- **¿Qué puedo reutilizar de la solución de otros problemas?:** El documento exige la implementación de al menos 9 patrones (3 de cada tipo). Basado en las pistas del texto, el reconocimiento lógico sería:
 - **Patrones Creacionales (Mínimo 3):**
 - Singleton (Obligatorio): Ideal para un GestorDeSesion que mantenga el usuario activo, o un SistemaPrincipal que centralice la coordinación de la plataforma.
 - Builder: La entidad Compra es muy compleja; involucra sumar entradas, configurar asientos, aplicar servicios adicionales y métodos de pago. Un CompraBuilder facilitaría su construcción paso a paso antes de confirmar el pago.
 - Factory Method : Para la creación de reportes. El sistema debe exportar en CSV o PDF. Una fábrica podría instanciar el generador de reportes adecuado según la petición del administrador.
 - **Patrones Estructurales (Mínimo 3):**
 - Decorator (Obligatorio) : Es la solución perfecta para el requisito de "Agregar servicios adicionales a la compra". A una EntradaBase se le pueden envolver dinámicamente decoradores como SeguroDecorator, VIPDecorator o ParqueaderoDecorator para sumar al precio final sin modificar la clase original.
 - Facade: El proceso de comprar entradas toca inventario (bloquear asiento), pagos y generación de tickets. Una fachada ProcesoCompraFacade simplificaría esta complejidad para la interfaz de usuario.
 - Proxy: Para el control de acceso. Solo el Administrador puede crear eventos o liberar asientos bloqueados. Un Proxy interceptaría estas peticiones validando el rol del usuario.
 - **Patrones de Comportamiento (Mínimo 3):**
 - Strategy (Obligatorio) : Ideal para el procesamiento de los "métodos de pago simulados". Se pueden tener estrategias como PagoTarjetaStrategy, PagoPSEStrategy, etc. También serviría para las diferentes "políticas

(cancelación, reembolso)".

- **State:** El documento es sumamente explícito con los estados. Un evento pasa por Borrador, Publicado, Pausado, etc. , y una compra por Creada, Pagada, Confirmada. El patrón State delegaría el comportamiento (por ejemplo, "no se pueden comprar entradas si el evento está Pausado") a clases de estado concretas.
- **Observer:** El sistema exige "recibir notificaciones sobre cambios de estado del evento y de sus compras". Los usuarios pueden suscribirse (Observers) a un evento (Subject) para ser notificados si se cancela.